



廣東工業大學
GUANGDONG UNIVERSITY OF TECHNOLOGY

嵌入式系统课程设计报告

《基于 STM32 的智能手表设计与实现》

姓 名： 沈雯洁

学 号： 3224009582

专 业： 微电子科学与工程

班 级： 微电子 1 班

队 名： 大天才智能手表

队 友： 何洁瑜，郑昭宜

指导老师： 周贤中



集成电路学院

2026 年 7 月 12 日

1 项目介绍

1.1 项目背景

本项目旨在利用嵌入式系统技术，设计并实现一款具备时间显示、计步和蓝牙通信功能的简易智能手表。项目选用意法半导体的 STM32F103C8T6 微控制器作为核心，搭配 OLED 显示屏、MPU6050 惯性测量单元、EC11 旋转编码器及 JDY-33 蓝牙模块，构建了一个集硬件驱动、多级菜单交互和无线数据传输于一体的软硬件系统。

1.2 主要功能概述

- 实时时钟显示：通过内部 RTC 提供时间与日期，支持通过菜单手动设置。
- 计步功能：利用 MPU6050 加速度计采集运动数据，经峰值检测算法实时统计步数。
- 蓝牙通信：通过 JDY-33 蓝牙模块将步数和时间信息发送至手机，实现数据远程查看。
- 多级菜单交互：旋转编码器实现页面切换、参数调整，按键确认与跳转，操作直观。
- 模块化设计：硬件上保留 I2C、USART 等接口，便于扩展；软件基于 FreeRTOS 多任务架构，提升实时性和可维护性。

2 项目设计目标

2.1 设计目标

- 完成 STM32 最小系统、电源、OLED、RTC、EC11 编码器的硬件连接与调试。
- 编写 OLED 驱动、RTC 时间读取、多级菜单逻辑，在 FreeRTOS 下创建显示任务和菜单交互任务。
- 实现计步数据采集与 OLED 显示，预留蓝牙传输接口。
- 验证蓝牙模块与手机的通信，能发送步数和时间数据。
- 形成一套完整的嵌入式开发文档和演示样品。

2.2 预期结果

- 上电后 OLED 正确显示当前时间、日期和步数。
- 旋转编码器可切换不同页面（时间主页、步数页、蓝牙发送页、设置页），按键确认进入子菜单或返回。
- 走动时步数实时更新，并通过蓝牙将数据发送到手机 APP。
- 系统稳定运行，无明显卡顿或重启。

3 项目设计与实现

3.1 硬件设计

3.1.1 系统架构

系统由主控模块、显示模块、传感器模块、人机交互模块、通信模块和电源模块六部分组成，结构框图如图 1 所示。

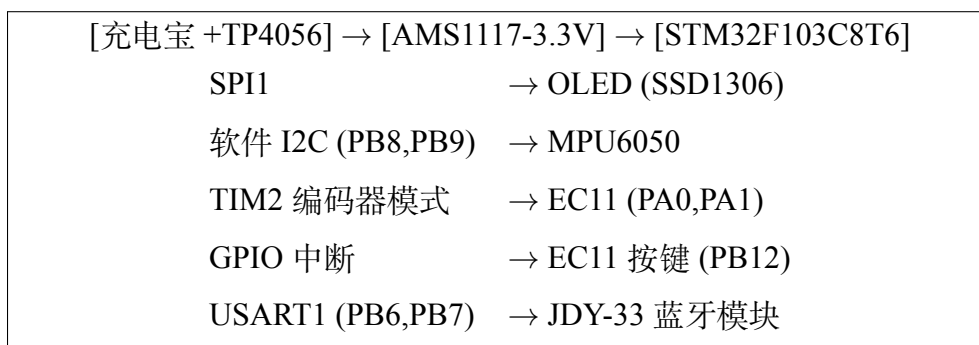


图 1 系统架构框图

3.1.2 关键模块选型

- 微控制器：STM32F103C8T6，Cortex-M3 内核，72MHz，64KB Flash，丰富的外设接口。
- 显示屏：0.96 寸 SPI OLED，驱动 IC 为 SSD1306，分辨率 128×64，单色显示。
- 传感器：MPU6050 六轴模块，使用其加速度计部分，I2C 接口。
- 旋转编码器：EC11 型，带按键，A/B 相输出用于方向判别，按键用于确认操作。
- 蓝牙模块：JDY-33 标准版，BLE 4.2，支持透传与 AT 指令，低功耗。
- 电源管理：TP4056 充电管理模块 + AMS1117-3.3V 线性稳压器，支持锂电池供电与 USB 充电。

3.1.3 电路原理图

STM32 引脚	连接元件	功能
PA5 (SPI1_SCK)	OLED SCK	SPI 时钟
PA7 (SPI1_MOSI)	OLED SDA	SPI 数据输出
PB0	OLED RES	复位控制
PB1	OLED DC	数据/命令选择
PA4	OLED CS	片选
PA0 (TIM2_CH1)	EC11 A 相	编码器输入
PA1 (TIM2_CH2)	EC11 B 相	编码器输入
PB12 (EXTI12)	EC11 按键	按键中断，内部上拉
PB8	MPU6050 SCL	软件 I2C 时钟
PB9	MPU6050 SDA	软件 I2C 数据
PB6 (US-ART1_TX)	JDY-33 RXD	蓝牙数据发送
PB7 (US-ART1_RX)	JDY-33 TXD	蓝牙数据接收
3.3V / GND	各模块电源	统一供电（蓝牙模块供电 5V）

表 1 关键引脚连接表

RTC 使用内部 LSI 振荡器，未外接晶振。

3.2 软件设计

3.2.1 软件架构

系统基于 FreeRTOS 实时操作系统，采用多任务并发执行的架构，主要任务包括：

1. 显示任务 (Task_Display, 优先级 2)：每 50ms 刷新一次 OLED 界面，根据菜单状态绘制相应内容。

2. 菜单交互任务 (Task_MenuCtrl, 优先级 1): 每 20ms 读取编码器增量和按键状态, 调用菜单状态机处理逻辑。
3. 计步任务 (Task_MPU6050, 优先级 3): 每 20ms 读取加速度数据, 执行计步算法, 更新全局步数变量。
4. 蓝牙任务 (Task_Bluetooth, 优先级 2): 每 2s 组装时间、步数字符串, 通过 USART1 发送给蓝牙模块。

任务间通过全局变量 (step_count, current_page 等) 和事件标志位进行通信, 共享资源通过 volatile 关键字和优先级安排避免竞态。

3.2.2 主程序流程

1. 系统上电, 初始化 HAL 库、时钟 (72MHz)、GPIO、SPI、软件 I2C、USART、RTC、编码器。
2. 初始化 OLED、菜单状态、MPU6050。
3. 创建四个 FreeRTOS 任务。
4. 启动调度器, 各任务并行运行。
5. 菜单状态机控制界面显示, 旋转编码器切换页面, 按键确认进入设置或返回。

3.2.3 关键代码片段与实现细节

1. OLED 驱动 (SSD1306 SPI 模式):

- 使用页寻址模式, 每页 8 像素, 逐页刷新 128 列。
- 为解决字体显示错位问题, 强制所有显示行坐标 y 为 8 的倍数, 并将字模字节进行位反转以适应 SSD1306 页内 bit0 映射到顶部的顺序。

2. 多级菜单状态机:

- 定义状态枚举 (主页面、主菜单、设置时/分/年/月/日、步数页、关于页)。
- Menu_Process 根据编码器增量移动光标或调整数值, 按键进入子状态或保存设置。
- Menu_Display 根据当前状态绘制不同界面, 所有文字 y 坐标对齐 8 的倍数。

3. 计步算法 (峰值检测):

- 读取三轴加速度, 计算幅值平方 $mag_sq = ax^2 + ay^2 + az^2$ 。
- 设置两个阈值: 高阈值 (静止值 + 约 6%) 和低阈值 (静止值 + 约 1.5%)。当 mag_sq 超过高阈值后进入“等待回落”状态, 回落至低阈值以下且满足最小步间隔 (250ms) 则计步一次。

- 完全采用整数运算，避免浮点。

4. 蓝牙数据发送：

- 使用 HAL_UART_Transmit 向 JDY-33 模块发送格式化字符串，包含步数、时间、日期，以 \r\n 结尾。
- 周期 2s，防止发送过快导致手机端数据堆积。

3.2.4 技术难点与解决方案

难点	解决方案
OLED 乱码	SSD1306 页内字节高低位与通用字模相反，通过 reverse_byte 函数解决；字符跨页撕裂通过强制 y 坐标 8 对齐解决。
MPU6050 硬件损坏，通信失败	更换新的 MPU6050，原使用硬件 I2C1 引脚映射错误 (PB6/PB7)，后改为 PB8/PB9 并采用软件模拟 I2C，解决了通信阻塞问题。
计步无反应	阈值设置不当，通过实时观察加速度平方值，最终设定高阈值 2.75 亿、低阈值 2.72 亿，可准确检测步伐。
蓝牙模块不工作	上电指示灯亮度微弱，排查问题为供电不足，后更改为 5V 供电

表 2 技术难点与解决方案

4 功能展示

4.1 功能描述

- 主页：显示时间（时: 分: 秒）、日期（年/月/日）和实时步数。
- 菜单：旋转编码器进入主菜单，可选择 “Set Time”、“Set Date”、“Steps”、“About”。在设置页可调整时间日期，步数页显示累计步数，关于页显示版本信息。
- 计步：实际走动时步数增加。
- 蓝牙：手机 APP 可连接蓝牙，接收数据。

4.2 演示结果

- OLED 显示正确，字符清晰，菜单切换流畅，按键响应灵敏。
- 时间设置功能正常，RTC 走时。
- 计步功能测试：将设备拿在手中来回摇晃，步数页面数字明显增长；正常走动时也能正确计步。
- 蓝牙连接成功，且手机 APP 可正常接收数据传输。

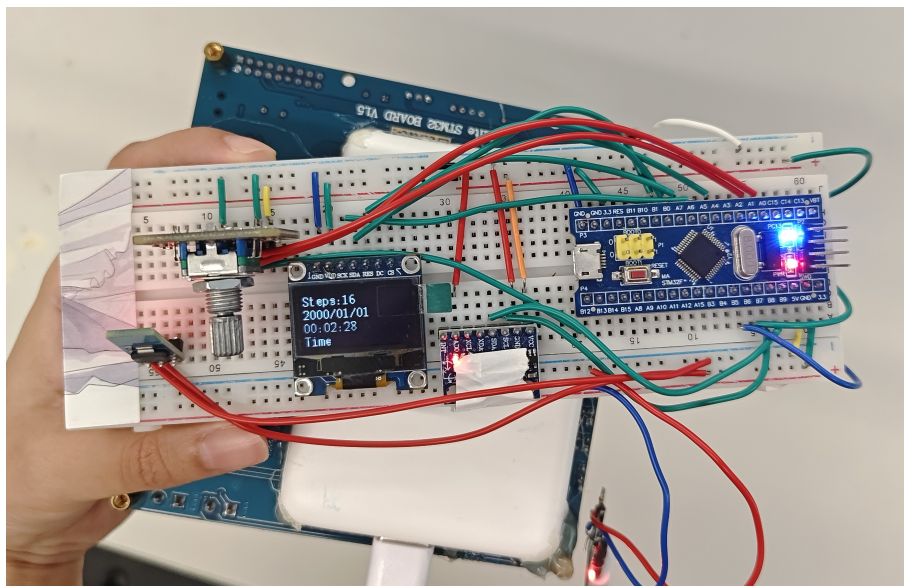


图 2 演示实物图 1

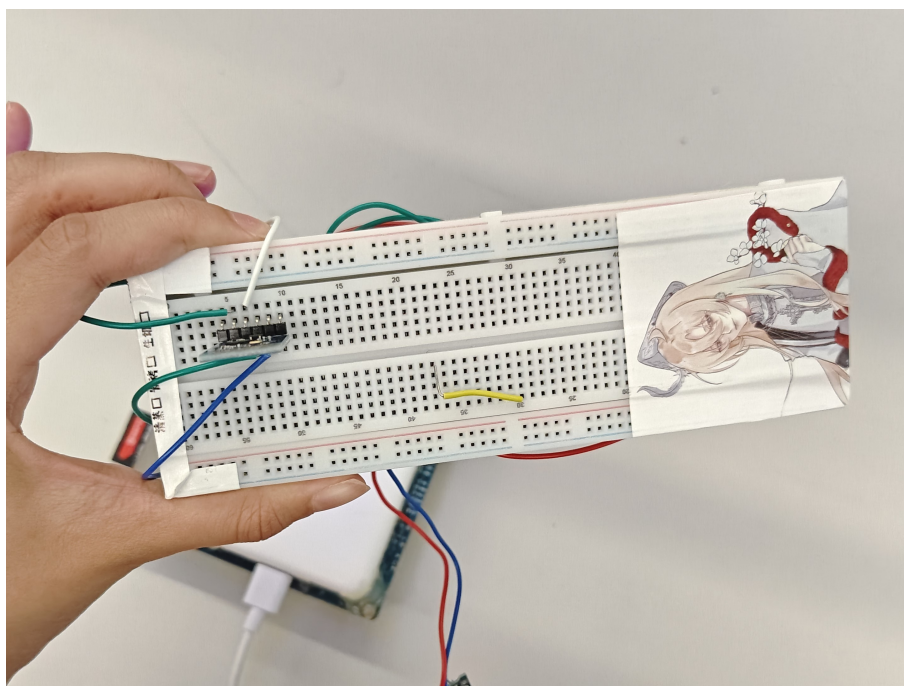


图 3 演示实物图 2

```
16:27:15 **** Incremental Build of configuration Debug for project MyWATCH ****
make -j20 all
arm-none-eabi-size MyWATCH.elf
   text    data     bss     dec      hex filename
   34828    112    14880    49820    c29c MyWATCH.elf
Finished building: default.size.stdout

16:27:16 Build Finished. 0 errors, 0 warnings. (took 1s.444ms)
```

图 4 调试成功截图

5 物料与成本

5.1 物料清单

名称	数量	单 价 (元)	用途	总价 (元)
STM32F103C8T6 最小系统板	1	12	主控制器	12
0.96 寸 SPI OLED 模块	1	15	信息显示	15
MPU6050 模块 (GY-521)	2	8	加速度采集	8
EC11 旋转编码器模块	1	3	人机交互	3
JDY-33 蓝牙模块	1	10	蓝牙通信	10
AMS1117-3.3V 稳压模块	1	2	电源转换	2
TP4056 充电保护模块	1	3	充电宝充电	3
杜邦线、排针、面包板等	若干	5	连接	5
总计				约 60 元

表 3 项目物料清单与成本

5.2 损耗与损坏记录

- 第一个 MPU6050 模块通信失败，反复调试无效，判断为损坏，更换后解决。
- JDY-33 模块上电指示灯微弱，更换更高电压后解决。
- 少量杜邦线插头变形，已替换。

6 个人贡献

6.1 报告撰写分工

小组报告中负责硬件部分、OLED 和 EC11 部分的编写，个人报告由本人独立完成

6.2 个人编码工作

- 编写并调试全部 OLED 驱动 (oled.c/oled.h)，包括字体库 fonts.h。
- 实现多级菜单状态机 (menu.c/menu.h)，支持旋转切换、按键确认和数值设置。
- 开发 EC11 编码器驱动 (encoder.c/encoder.h) 及按键中断处理。
- 协助成员调试 MPU6050 驱动 (mpu6050.c/mpu6050.h) 并排查问题，完成软件 I2C 和计步算法的调试，并将数据接入 OLED 显示中。
- 在 main.c 中集成 FreeRTOS 任务创建与管理。

6.3 硬件连接工作

- 完成全部模块的接线，包括最小系统、电源、OLED、编码器、传感器、蓝牙的互联。
- 使用万用表进行信号测量与故障排查。
- 确认并修正 I2C 引脚错误 (PB8/PB9 替代 PB6/PB7)，保证了传感器通信。

6.4 开发中遇到的问题及解决方案

详见表 2。

7 项目成果与反思

7.1 成果展示

- 成功搭建了基于 STM32 的智能手表硬件平台，各模块工作基本正常。
- OLED 显示效果良好，菜单交互流畅，时间日期可手动设置。
- 计步算法成功运行，能根据摇晃和走动准确计步，并实时显示在屏幕上。
- FreeRTOS 多任务调度稳定，无死机或重启现象。
- 蓝牙模块已能与手机配对，数据发送成功。

7.2 项目评估

成功之处:

- 深入理解了 SSD1306 的驱动机制和 SPI 通信, 解决了字体显示难题。
- 掌握了 I2C 时序和软件模拟方法, 提高了硬件调试能力。
- 多级菜单状态机的设计清晰, 易于扩展。
- FreeRTOS 的应用使系统响应及时, 结构分明。
- 计步算法经过实际测试, 灵敏度和准确性满足预期。
- 成功驱动蓝牙模块, 与手机连接成功, 且能正常传输数据。

不足之处:

- RTC 使用内部 LSI 导致走时存在误差, 未及时外接晶振。
- 项目时间有限, 部分调试不够深入, 如低功耗优化、PCB 设计等未涉及。

7.3 个人反思与改进方向

- 硬件设计阶段应更加仔细地核对引脚功能, 避免因映射错误导致大量调试时间浪费。
- 传感器初始化时序需严格遵循数据手册, 对于复位、唤醒等关键步骤应给予足够的延时。
- 未来可绘制 PCB 板, 将各模块集成, 减少飞线, 提高可靠性。
- 可加入电池电量检测、自动校准、步数清零等功能, 提升用户体验。
- 加强软件架构的模块化, 例如将菜单系统独立成库, 便于复用。

8 附录

8.1 设计图纸

项目硬件接线示意图和系统框图可参考图 1 及表 1。

8.2 完整代码清单

项目工程文件结构:

MyWatch/

Core

Inc

```
fonts.h
oled.h
encoder.h
menu.h
mpu6050.h
soft_i2c.h
```

Src

```
main.c
oled.c
encoder.c
menu.c
mpu6050.c
soft_i2c.c
```

Drivers

... (STM32Cube FW)

SmartWatch.ioc

以下列出部分关键代码段。

8.2.1 OLED 驱动关键代码 (oled.c)

```
1 // 引脚控制宏
2 #define OLED_RST_LOW()    HAL_GPIO_WritePin(GPIOB, GPIO_PIN_0,
   GPIO_PIN_RESET)
3 #define OLED_RST_HIGH()   HAL_GPIO_WritePin(GPIOB, GPIO_PIN_0,
   GPIO_PIN_SET)
4 #define OLED_DC_LOW()     HAL_GPIO_WritePin(GPIOB, GPIO_PIN_1,
   GPIO_PIN_RESET)
5 #define OLED_DC_HIGH()    HAL_GPIO_WritePin(GPIOB, GPIO_PIN_1,
   GPIO_PIN_SET)
6 #define OLED_CS_LOW()     HAL_GPIO_WritePin(GPIOA, GPIO_PIN_4,
   GPIO_PIN_RESET)
7 #define OLED_CS_HIGH()    HAL_GPIO_WritePin(GPIOA, GPIO_PIN_4,
   GPIO_PIN_SET)
```

```
8
9 static uint8_t OLED_GRAM[128][8]; // 128×64 缓冲区
10
11 // 字节反转, 适配 SSD1306 页内 bit0 = 顶部
12 static uint8_t reverse_byte(uint8_t b) {
13     b = ((b & 0xF0) >> 4) | ((b & 0x0F) << 4);
14     b = ((b & 0xCC) >> 2) | ((b & 0x33) << 2);
15     b = ((b & 0xAA) >> 1) | ((b & 0x55) << 1);
16     return b;
17 }
18
19 // 写命令/数据
20 static void OLED_WriteByte(uint8_t dat, uint8_t cmd) {
21     OLED_CS_LOW();
22     if (cmd == 0) OLED_DC_LOW(); // 命令
23     else          OLED_DC_HIGH(); // 数据
24     HAL_SPI_Transmit(&hspi1, &dat, 1, 10);
25     OLED_CS_HIGH();
26 }
27
28 // SSD1306 初始化序列
29 void OLED_Init(void) {
30     OLED_RST_HIGH(); HAL_Delay(100);
31     OLED_RST_LOW();  HAL_Delay(100);
32     OLED_RST_HIGH(); HAL_Delay(100);
33
34     OLED_WriteByte(0xAE, 0); // 关闭显示
35     OLED_WriteByte(0x20, 0); OLED_WriteByte(0x00, 0); // 水平寻址
36     OLED_WriteByte(0xA1, 0); // 段重映射 (左右方向)
37     OLED_WriteByte(0xC8, 0); // COM 扫描方向 (上下方向)
38     OLED_WriteByte(0xA6, 0); // 正常显示
39     OLED_WriteByte(0xA8, 0); OLED_WriteByte(0x3F, 0); // 64 路复用
40     OLED_WriteByte(0x8D, 0); OLED_WriteByte(0x14, 0); // 电荷泵开
41     OLED_WriteByte(0xAF, 0); // 开显示
```

```
42     OLED_Clear();
43     OLED_Update();
44 }
45
46 // 清屏
47 void OLED_Clear(void) {
48     memset(OLED_GRAM, 0x00, sizeof(OLED_GRAM));
49 }
50 void OLED_ClearBuffer(void) { OLED_Clear(); }
51
52 // 将缓冲区刷新到 OLED
53 void OLED_Update(void) {
54     for (uint8_t page = 0; page < 8; page++) {
55         OLED_WriteByte(0xB0 + page, 0); // 页地址
56         OLED_WriteByte(0x00, 0);        // 列低
57         OLED_WriteByte(0x10, 0);        // 列高
58         for (uint8_t x = 0; x < 128; x++) {
59             OLED_WriteByte(OLED_GRAM[x][page], 1);
60         }
61     }
62 }
63
64 // 显示 8×16 字符 (y 必须是 8 的倍数)
65 void OLED_ShowChar(uint8_t x, uint8_t y, char ch, uint8_t size) {
66     if (x + 8 > 128 || y + 16 > 64) return;
67     uint8_t index = ch - ' '; // 字体偏移
68     uint8_t page = y / 8;
69     for (uint8_t col = 0; col < 8; col++) {
70         uint8_t upper = Font_8x16[index][col];
71         uint8_t lower = Font_8x16[index][col + 8];
72         OLED_GRAM[x + col][page] = reverse_byte(upper);
73         OLED_GRAM[x + col][page + 1] = reverse_byte(lower);
74     }
75 }
```

```
76
77 // 显示字符串
78 void OLED_ShowString(uint8_t x, uint8_t y, const char *str, uint8_t
    size) {
79     while (*str) {
80         OLED_ShowChar(x, y, *str++, size);
81         x += 8;
82         if (x + 8 > 128) { x = 0; y += 16; }
83     }
84 }
85
86 // 格式化输出
87 void OLED_Printf(uint8_t x, uint8_t y, const char *fmt, ...) {
88     char buf[64];
89     va_list args;
90     va_start(args, fmt);
91     vsnprintf(buf, sizeof(buf), fmt, args);
92     va_end(args);
93     OLED_ShowString(x, y, buf, 1);
94 }
```

Listing 1: OLED 初始化、更新与字符显示

8.2.2 字体库 (fonts.h 部分)

```
1 static const uint8_t Font_8x16[][16] = {
2     // ' '
3     {0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00},
4     // '0'
5     {0x00,0xE0,0x10,0x08,0x08,0x10,0xE0,0x00,0x00,0x0F,0x10,0x20,0x20,0x10,0x0F,0x00},
6     // '1'
7     {0x00,0x10,0x10,0xF8,0x00,0x00,0x00,0x00,0x00,0x20,0x20,0x3F,0x20,0x20,0x00,0x00},
```

```
8 // '2'
9 {0x00,0x30,0x08,0x08,0x08,0x88,0x70,0x00,0x00,0x20,0x30,0x28,0x24
   ,0x22,0x21,0x00},
10 // 'A'
11 {0x00,0x00,0xE0,0x18,0x18,0xE0,0x00,0x00,0x00,0x3E,0x09,0x08,0x09
   ,0x3E,0x00,0x00},
12 // 'B'
13 {0x00,0xF8,0x88,0x88,0x88,0x70,0x00,0x00,0x00,0x3F,0x21,0x21,0x21
   ,0x1E,0x00,0x00},
14 // 'C'
15 {0x00,0xE0,0x10,0x08,0x08,0x08,0x30,0x00,0x00,0x0F,0x10,0x20,0x20
   ,0x20,0x10,0x00},
16 // ':'
17 {0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x0C,0x0C,0x00
   ,0x00,0x00,0x00},
18 // '/'
19 {0x00,0x00,0x00,0x00,0x80,0x60,0x18,0x00,0x00,0x30,0x0C,0x03,0x00
   ,0x00,0x00,0x00},
20 // ... 完整字体包含 95 个 ASCII 字符, 此处省略其余
21 };
```

Listing 2: 8×16 纵向取模字体 (数字、字母示例)

8.2.3 软件 I2C 驱动 (soft_i2c.c 部分)

```
1 #define SOFT_I2C_SCL_PIN    GPIO_PIN_8
2 #define SOFT_I2C_SDA_PIN    GPIO_PIN_9
3 #define SOFT_I2C_PORT       GPIOB
4
5 void Soft_I2C_Init(void) {
6     // 将 PB8,PB9 配置为开漏输出, 并置高
7     SDA_Output(); HAL_GPIO_WritePin(SOFT_I2C_PORT, SOFT_I2C_SDA_PIN,
   GPIO_PIN_SET);
8     SCL_Output(); HAL_GPIO_WritePin(SOFT_I2C_PORT, SOFT_I2C_SCL_PIN,
   GPIO_PIN_SET);
```

```
9 }
10
11 static void delay_us(uint32_t us) {
12     for (volatile uint32_t i = 0; i < us * 20; i++) __NOP();
13 }
14
15 static void I2C_Start(void) { /* 起始信号 */ }
16 static void I2C_Stop(void) { /* 停止信号 */ }
17
18 uint8_t Soft_I2C_ReadBytes(uint8_t addr, uint8_t reg, uint8_t *data,
19     uint8_t len) {
20     I2C_Start();
21     if (I2C_WriteByte(addr << 1)) { I2C_Stop(); return 1; }
22     if (I2C_WriteByte(reg)) { I2C_Stop(); return 2; }
23     I2C_Start();
24     if (I2C_WriteByte((addr << 1) | 0x01)) { I2C_Stop(); return 3; }
25     for (uint8_t i = 0; i < len; i++) {
26         data[i] = I2C_ReadByte(i == (len - 1) ? 1 : 0);
27     }
28     I2C_Stop();
29     return 0;
30 }
31 // 详细实现见工程文件
```

Listing 3: 软件 I2C 读写函数

8.2.4 菜单状态机 (menu.c 片段)

```
1 void Menu_Process(int8_t enc_diff, uint8_t btn_pressed) {
2     switch (state) {
3         case STATE_MAIN_PAGE:
4             if (enc_diff != 0) { state = STATE_MAIN_MENU; menu_cursor
5                 = 0; }
6             break;
7         case STATE_MAIN_MENU:
```

```
7         if (enc_diff > 0) menu_cursor = (menu_cursor + 1) % 4;
8         else if (enc_diff < 0) menu_cursor = (menu_cursor + 3) %
9             4;
10        if (btn_pressed) {
11            switch (menu_cursor) {
12                case 0: state = STATE_SET_HOUR; break;
13                case 1: state = STATE_SET_YEAR; break;
14                case 2: state = STATE_STEP; break;
15                case 3: state = STATE_ABOUT; break;
16            }
17        }
18        break;
19        // ... 时间/日期设置状态机略
20        case STATE_SET_HOUR:
21            temp_hour = (temp_hour + enc_diff) % 24;
22            if (btn_pressed) state = STATE_SET_MINUTE;
23            break;
24        // ... 其他类似
25    }
26
27    void Menu_Display(void) {
28        OLED_ClearBuffer();
29        RTC_TimeTypeDef sTime;
30        RTC_DateTypeDef sDate;
31        HAL_RTC_GetTime(&hrtc, &sTime, RTC_FORMAT_BIN);
32        HAL_RTC_GetDate(&hrtc, &sDate, RTC_FORMAT_BIN);
33        switch (state) {
34            case STATE_MAIN_PAGE:
35                OLED_ShowString(0, 0, "Time", 1);
36                OLED_Printf(0, 16, "%02d:%02d:%02d", sTime.Hours, sTime.
37                    Minutes, sTime.Seconds);
38                OLED_Printf(0, 32, "%02d/%02d/%02d", sDate.Year, sDate.
39                    Month, sDate.Date);
```

```
38         OLED_Printf(0, 48, "Steps:%lu", step_count);
39         break;
40     case STATE_STEP:
41         OLED_ShowString(0, 0, "Steps:", 1);
42         OLED_Printf(0, 16, "%lu", step_count);
43         OLED_ShowString(0, 48, "Press to ret", 1);
44         break;
45     // ... 其他页面
46 }
47 OLED_Update();
48 }
```

Listing 4: 菜单处理与显示

8.2.5 计步与蓝牙任务 (main.c 片段)

```
1 // 计步任务
2 void Task_MPU6050(void *argument) {
3     int16_t ax, ay, az;
4     const int32_t THRESH_HIGH_SQ = 275000000;
5     const int32_t THRESH_LOW_SQ  = 272000000;
6     const TickType_t MIN_STEP_INTERVAL = pdMS_TO_TICKS(250);
7     uint8_t step_state = 0;
8     uint32_t last_tick = 0;
9     for (;;) {
10         if (MPU6050_ReadAccel(&ax, &ay, &az) != HAL_OK) {
11             vTaskDelay(pdMS_TO_TICKS(20));
12             continue;
13         }
14         int32_t mag_sq = (int32_t)ax*ax + (int32_t)ay*ay + (int32_t)
            az*az;
15         if (mag_sq > THRESH_HIGH_SQ && step_state == 0) step_state =
            1;
16         if (mag_sq < THRESH_LOW_SQ && step_state == 1) {
17             if ((xTaskGetTickCount() - last_tick) >=
```

```
MIN_STEP_INTERVAL) {  
18     step_count++;  
19     last_tick = xTaskGetTickCount();  
20 }  
21     step_state = 0;  
22 }  
23     vTaskDelay(pdMS_TO_TICKS(20));  
24 }  
25 }  
26  
27 // 蓝牙发送任务 (周期 2 秒)  
28 void Task_Bluetooth(void *argument) {  
29     char msg[64];  
30     RTC_TimeTypeDef sTime;  
31     RTC_DateTypeDef sDate;  
32     for (;;) {  
33         HAL_RTC_GetTime(&hrtc, &sTime, RTC_FORMAT_BIN);  
34         HAL_RTC_GetDate(&hrtc, &sDate, RTC_FORMAT_BIN);  
35         sprintf(msg, "Steps:%lu Time:%02d:%02d:%02d Date:20%02d-%02d  
-%02d\r\n",  
36             step_count,  
37             sTime.Hours, sTime.Minutes, sTime.Seconds,  
38             sDate.Year, sDate.Month, sDate.Date);  
39         HAL_UART_Transmit(&huart1, (uint8_t*)msg, strlen(msg), 1000);  
40         vTaskDelay(pdMS_TO_TICKS(2000));  
41     }  
42 }
```

Listing 5: 计步任务与蓝牙发送任务